VOLCANO-CA-MC: DOKUMENTACJA TECHNICZNA

Jak to działa: architektura i kluczowe miejsca w kodzie

Bartosz Pajda

Wygenerowano: 27.10.2026

Ten przewodnik prowadzi przez cały projekt od strony „jak to jest zrobione i dlaczego": co symulujemy, gdzie co leży, jak jedna symulacja przechodzi od pliku konfiguracyjnego do pliku z wynikiem, jak działa GUI i automatyzacja, co pokazał profil i odpowiedzi na najczęstsze pytania. Numery linii dotyczą stanu z 21.10.2026 (po wersji MPI); przy zmianach kodu mogą się przesunąć o kilka linii, nazwy funkcji zostają. Jak wszystko uruchomić: **dokument „Instalacja, budowanie i uruchamianie"**.

1. Co symuluje projekt (w jednym akapicie)

Symulujemy erupcję wulkanu w przestrzeni z sześciennych komórek (wokseli) 2D albo 3D. Każda komórka jest powietrzem, skałą, płynną lawą albo bazaltem (zastygłą lawą). **Automat komórkowy (CA)** w trybie `lava` odpowiada klasycznemu „rozrostowi ziaren": **kratery to zarodki** (każdy ma własny numer `flowId`, jak ziarno), a **lawa to rosnące ziarna**: w każdym kroku komórka powietrza obok lawy może stać się lawą i dostaje numer krateru, z którego lawa przyszła (najczęstszy wśród sąsiadów, jak w rozroście ziaren). Różnica względem klasycznego rozrostu: lawa ma temperaturę, spada w dół, płynie w kierunku największego spadku, stygnie i poniżej temperatury zastygania zamienia się w bazalt. Tryb `classic` to zwykły rozrost ziaren bez wulkanu (do porównań wydajności). Drugi algorytm, **Monte Carlo (model Pottsa)**, odpowiada naiwnemu „rozrostowi ziaren MC": zastygły bazalt krystalizuje, a każda komórka ma orientację kryształu 1…Q, a losowe próby zmiany orientacji akceptujemy, gdy nie zwiększają energii granic ziaren; długo płynna lawa ma większą „mobilność", więc tworzy duże kryształy (tryb `basalt`), a tryb `full` to klasyczny naiwny rozrost MC na całej przestrzeni (do porównania czasów CA vs MC).

2. Mapa projektu

```
volcano-ca-mc/
├── CMakeLists.txt, CMakePresets.json   budowanie: presety msvc-release/-debug, gcc-release/-
debug, presety profilu
├── engine/                             SILNIK C++17, „wersja serwerowa" bez GUI
│   ├── src/main.cpp                    punkt wejścia: wybór trybu (symulacja, --batch, --
benchmark, --validate…), kody wyjścia
│   ├── src/app/                        „klej" programu
│   │   ├── Cli.*                       parser argumentów (--config, --set, --out, --mc-from,
--batch…)
│   │   ├── Simulation.*                jedna symulacja: teren → kratery → CA → (MC) → zapis
+ times.json
│   │   ├── McStage.*                   faza Monte Carlo: przygotowanie, pętla MCS,
statystyki, zapis
│   │   ├── Batch.*                     engine --batch: seria configów z folderu,
batch_summary.csv
│   │   ├── Benchmark.*                 engine --benchmark: vector vs macierz, zgodność
wyników
│   │   ├── ConfigCheck.*               --print-defaults i --validate (dla GUI)
│   │   └── Progress.hpp                linie „PROGRESS …" dla GUI (--progress)
│   ├── src/core/                       podstawy
│   │   ├── Types.hpp                   typy komórek, wymiary, Microstructure (wynik bez
ramki)
│   │   ├── Storage.hpp                 dwie reprezentacje siatki: VectorStorage (płaska) i
MatrixStorage (vector³)
│   │   ├── Neighborhood.*              sąsiedztwa jako listy przesunięć (VN, Moore,
pentagonal, hexagonal)
│   │   ├── Boundary.*                  warunki brzegowe = wypełnianie ramki 1 komórki
│   │   ├── Rng.hpp                     hashRand: losowość z położenia komórki (bez stanu)
│   │   ├── Timer.hpp                   pomiar czasu (steady_clock)
│   │   ├── ProcessInfo.*, MemoryModel.hpp  pamięć procesu i model teoretyczny (profil
aplikacji)
│   │   └── ChildProcess.*              uruchamianie procesu potomnego (dla --batch)
│   ├── src/terrain/                    teren: stożek z szumem, mapa wysokości z CSV, poziomy
przelewu, kratery
│   ├── src/ca/                         automat: LavaCA.hpp (szablon po Storage), CaRules.hpp
(opis reguł!)
│   ├── src/mc/                         Monte Carlo: PottsMC.*, Grains.* (liczenie ziaren),
CoolingMobility.*, McRules.hpp
│   ├── src/io/                         Config (wczytanie + walidacja), ResultWriter/Reader
```

```
(txt, bin), TimingReport (times.json)
│   ├── src/parallel/                    OpenMP: Omp.hpp (ompFor, ustawienia), Affinity.*
(rdzenie P), ThreadStats.*;
│   │                                    Strip.* (MPI): podział warstw na pasy, plan warstw
brzegowych, interfejs StripLink
│   ├── src/mpi/                         MPI: main_mpi.cpp (program engine_mpi),
MpiSimulation.* (rozsyłanie, pasy,
│   │                                    zebranie wyników), MpiHalo.* (wymiana warstw
brzegowych, MPI_Allreduce);
│   │                                    typy MPI i MPI-IO: MpiTypes.* (typy pochodne: blok
configu, kratery), MpiIo.* (zapis i odczyt MPI-IO)
│   ├── tests/                           testy jednostkowe doctest (m.in. test_front.cpp,
test_parallel.cpp, test_strip.cpp,
│   │                                    test_replay.cpp); tests/mpi/: testy engine_mpi pod
mpiexec (1-8 procesów)
│   └── third_party/                     json.hpp (nlohmann), doctest.h: jedyne zależności C++
├── gui/                                 GUI web: backend Python/FastAPI + frontend HTML/JS
│   ├── __main__.py                      python -m gui: start serwera na 127.0.0.1:8000 i
przeglądarki
│   ├── app.py                           endpointy /api/… (formularz, zadania, kolejka, serie,
wyniki, porównanie, eksport)
│   ├── engine.py                        wywołania silnika: --print-defaults, --validate,
polecenie uruchomienia
│   ├── jobs.py                          JobManager: zadania, kolejka, workery, serie,
przypięcie, jobs.json
│   ├── series.py                        generator serii (wspólny z tools/make_series.py),
szablony
│   ├── compare.py                       tabela porównania wyników serii, CSV
│   ├── results.py, render.py            odczyt wyników (bin/txt, memmap), przekroje, ściany,
powierzchnia, woksele
│   ├── heightmap.py, export.py          mapy wysokości (CSV/PNG, metadane geo), eksport ZIP i
VTK
│   ├── replay.py                        odtwarzanie erupcji: stan w kroku k z
arrivalStep/solidStep
│   ├── static/                          frontend: index.html, css/, js/ (main, form, series,
jobs, compare, viewer2d, viewer3d, replay, ventpicker,
│   │                                    perf: zakładka „Wydajność”…), data/wydajnosc.json
(dane zakładki „Wydajność”), vendor/ (three.js, Chart.js)
│   └── tests/                           pytest (146 testów), screenshots.py (zrzuty),
run_ca_vs_mc.py (pomiar CA vs MC przez GUI)
├── tools/                               narzędzia bez GUI
│   ├── plot_slice.py, plot_mc_stats.py  przekrój PNG i wykres statystyk MC
│   ├── make_series.py                   seria configów + engine --batch
│   ├── export_vtk.py                    wynik → .vti dla ParaView
│   ├── make_heightmaps.py               przykładowe mapy wysokości
│   ├── fetch_dem.py                     prawdziwy teren: kafelki AWS Terrain Tiles →
configs/heightmaps/<nazwa>.csv/.png/.json
│   ├── latlon_to_grid.py                (szer., dł.) ↔ kolumna (x, y) siatki mapy z
prawdziwego terenu
│   └── wydajnosc_gui.py                 dane zakładki „Wydajność”: bench/results/ (pomiary
wersji szybkich i równoległych) → gui/static/data/wydajnosc.json
├── bench/                               profil aplikacji: run_profile.py, plot_profile.py,
profile_gcc.sh, profile_msvc.py, results/, profiles/
│                                        optymalizacja sekwencyjna: ab.py (test A/B),
compare_results.py (zgodność bitowa), plot_optymalizacja.py, results/ab/
│                                        OpenMP: scaling.py (skalowanie OpenMP),
mc_validation.py, plot_openmp.py, micro/omp_overhead.cpp, results/omp/
│                                        MPI: scaling_mpi.py (skalowanie MPI), plot_mpi.py,
compare_results.py --launch, results/mpi/
│                                        typy MPI i MPI-IO: mpi_io.py (typy pochodne i MPI-
IO), plot_mpi_io.py, results/mpi_io/
│                                        testy wydajności: zestawienie.py, memory.py,
cachegrind.py, cache_info.py, cell_time.py,
│                                        scaling*.py --weak, micro/bandwidth.cpp,
plot_testy.py, results/testy/
├── configs/                             przykładowe configi (test_2d, test_3d, etna_3d,
classic_ref, lava_mc_*, mc_full_*, heightmap_*,
│                                        etna_real, etna_real_boczna, wezuwiusz_real),
heightmaps/ (mapy CSV + PNG + JSON, README z przypisaniem
│                                        danych); gui/ i series/: generowane (poza
repozytorium)
```

```
├── results/                          wyniki (poza repozytorium); results/gui/: zadania i
serie z GUI
└── docs/                             dokumenty tematyczne *.md (spis: docs/README.md),
przewodniki (architektura, uruchamianie), pdf/, raporty raport-1 i raport-2, img/, data/
```

3. Droga jednej symulacji: od `engine --config` do pliku wynikowego

Przykład: `build\Release\engine.exe --config configs\lava_mc_3d.json --out results\lava_mc_3d`.

Krok 1. `main` i argumenty

Co się dzieje: program startuje stoper całego przebiegu, analizuje argumenty i wybiera tryb: pomoc, domyślny config, walidacja, benchmark, seria (`--batch`) albo zwykła symulacja. Wyjątki zamienia na kody wyjścia (1: błąd configu, 2: błąd argumentów, 3: inny błąd, 5: seria z błędem).

Gdzie: `engine/src/main.cpp`, `main()` (linie 20–56); parser `engine/src/app/Cli.cpp`, `Cli::parse()` (linie 40–129): każda opcja to gałąź `if`, a na końcu sprawdzane są niedozwolone połączenia (np. `--batch` z `--config`).

```
const CliOptions opt = Cli::parse(argc, argv);
...
if (opt.benchmark) {
    return Benchmark::runFromCli(opt);
}
// Etap 5: seria configów z folderu, każdy w osobnym procesie silnika.
if (!opt.batchDir.empty()) {
    return BatchRunner::runFromCli(opt);
}
return Simulation::runFromCli(opt, sinceStart);
```

`--set klucz=wartość` jest zbierane do listy `overrides`, a `--grid matrix` to skrót `--set grid=matrix`. `--mc-from <folder>` przełącza na „samo MC na wyniku CA z pliku", `--batch <folder>` na serię (punkt 5), `--threads N` to skrót `--set parallel.threads=N` (wersja OpenMP, krok 6a), a `--thread-stats` włącza pomiar pracy wątków.

Krok 2. Wczytanie i walidacja configu

Co się dzieje: zaczynamy od wartości domyślnych (pełny JSON wpisany w kod), nakładamy na nie plik JSON, a potem kolejne `--set`. Nieznany klucz to błąd (łapie literówki), klucze zaczynające się od `_` są pomijane (komentarze). Potem każdy parametr jest sprawdzany (zakres, typ, dozwolone wartości, połączenia, np. lawa 2D musi mieć `ny = 1`) i trafia do struktury `Config`. Błędy mają polskie komunikaty z nazwą parametru.

Gdzie: `engine/src/io/Config.cpp`: `Config::defaultsJson()` (153), `Config::merge()` (178–203), `Config::applyOverride()` (205–235), `Config::load()` (237–266), `Config::fromJson()` (268–437), `Config::parallelSettings()` (463–469; sekcja `parallel` → `ParallelSettings`, `threads = 0` → `omp_get_max_threads()`); warianty MPI (`mpi.exchange`, `mpi.pack`, `mpi.bcast`, `mpi.io`): tylko walidacja w `fromJson()`, odczyt metodami `mpiExchangeChanged()`, `mpiPackDatatype()`, `mpiBcastStruct()`, `mpiIoParallel()` (447–461).

```
Config Config::load(const std::string& path, const std::vector<std::string>& overrides) {
    ojson j = defaultsJson();
    ...
        file = ojson::parse(in, nullptr, true, true);   // komentarze // i /* */ dozwolone
        merge(j, file, "");                             // nieznany klucz -> ConfigError
    ...
    for (const std::string& o : overrides) {
        applyOverride(j, o);                            // "space.nx=100" -> {"space": {"nx":
100}}
    }
    Config c = fromJson(j);                             // walidacja wszystkich pól
```

Pełny config po scaleniu (`cfg.effective`) zapisujemy jako `config_used.json`; wiadomo dokładnie, z czym liczył przebieg. Czas tej fazy to sekcja `readConfig` w `times.json` (`Simulation::runFromCli`, linia ~536).

Krok 3. Alokacja siatki: VectorStorage vs MatrixStorage, ramka, SoA

Co się dzieje: pełny przebieg automatu (`ca.sweep = full`) ma **dwa bufory** siatki (`cur`: stan bieżący, `next`: następny); domyślny automat frontalny (`ca.sweep = front`, z optymalizacji sekwencyjnej) ma **jeden** bufor i listy zmian (krok 6). Każde pole komórki to **osobna tablica** (SoA, *structure of arrays*): `type` (1 B), `flowId` (4 B), `temp` (4 B), `solidStep` i `arrivalStep` (po 2 B), razem 13 B na komórkę w buforze. Dookoła siatki jest **ramka 1 komórki** w osiach używanych przez sąsiedztwo (w 2D tylko w dwóch osiach płaszczyzny: `Dims::frameX/Y/Z`, optymalizacja sekwencyjna), więc sąsiad komórki z brzegu zawsze istnieje, a warunki brzegowe to wypełnienie ramki (krok 6). Dwie reprezentacje: `VectorStorage`: jedna płaska `std::vector` na pole; `MatrixStorage`: `vector<vector<vector<T>>>` (porównanie dwóch reprezentacji siatki). Automat sięga do komórek przez **pozycję** `Storage::Pos` (VectorStorage: indeks w tablicy z ramką, MatrixStorage: współrzędne): sąsiad = pozycja + przesunięcie liniowe `linear(o) = dx + sx·(dy + sy·dz)` policzone raz w konstruktorze automatu, następna komórka w osi x = `nextX(p)`. Obie reprezentacje mają ten sam interfejs, a automat jest **szablonem** `LavaCA<Storage>`: wybór następuje raz, bez funkcji wirtualnych w gorącej pętli.

Gdzie: `engine/src/core/Storage.hpp`: `VectorStorage` (43–176), `linear()` (100–104), `index()` (160–165), `MatrixStorage` (183–291); wybór w `engine/src/app/Simulation.cpp`, `Simulation::run()` (294–332), alokacja w `Simulation::runWith()` (159–292).

```
// VectorStorage: indeks w płaskiej tablicy z ramką (tylko w osiach używanych); base_ =
indeks komórki (0, 0, 0).
std::size_t index(int x, int y, int z) const {
    assert(x >= -dims_.frameX() && x < dims_.nx + dims_.frameX() && y >= -dims_.frameY() &&
           y < dims_.ny + dims_.frameY() && z >= -dims_.frameZ() && z < dims_.nz +
dims_.frameZ());
    return base_ + static_cast<std::size_t>(x) +
           sx_ * (static_cast<std::size_t>(y) + sy_ * static_cast<std::size_t>(z));
}

// Simulation::run — wybór reprezentacji raz, dalej kod szablonowy
s = (cfg_.grid == GridKind::Matrix) ? runWith<MatrixStorage>(outDir, timing)
                                    : runWith<VectorStorage>(outDir, timing);
```

W pełnym przebiegu po kroku bufory są zamieniane `swap()`: zamiana wskaźników, bez kopiowania danych (O(1)); automat frontalny zapisuje zmiany wprost do `cur` po regułach całego kroku (krok 6).

Krok 4. Teren: stożek albo mapa wysokości, wypełnianie zagłębień

Co się dzieje: teren to **mapa wysokości**: dla każdej kolumny (x, y) liczba komórek skały od dna. Generator robi stożek `h = peakHeight · (1 - r/baseRadius)` z płaskodennym kraterem w środku i gładkim szumem (szum wartości z hasha, 3 oktawy; biały szum robiłby jamy, w których lawa grzęźnie). Zamiast stożka można podać mapę z pliku CSV (`terrain.type = file`; w GUI także import CSV/PNG/JPG). Potem liczony jest **poziom przelewu** każdej kolumny algorytmem **priority-flood** (Barnes i in. 2014): „zalewamy” mapę od brzegów w kolejności rosnącej wysokości, a każda kolumna dostaje max(własna wysokość, poziom, z którego przyszła woda). Wynik mówi, do jakiej wysokości wypełni się krater, zanim ciecz wyleje się przez najniższy punkt krawędzi; z tego korzysta reguła wypełniania zagłębień (krok 6).

Gdzie: `engine/src/terrain/TerrainGenerator.cpp`: `TerrainGenerator::cone()` (61–84), `smoothNoise()`, `fillDepressions()` (86–161, kolejka priorytetowa); `engine/src/terrain/HeightmapLoader.cpp`, `loadCsv()` (37–83); wywołanie: `Simulation::buildTerrain()` (138–146) i `LavaCA::applyTerrain()` (`engine/src/ca/LavaCA.hpp`, 185–211; wpisanie skały do siatki: w wielu wątkach po warstwach, wersja OpenMP).

```
for (int y = 0; y < d.ny; ++y) {
    for (int x = 0; x < d.nx; ++x) {
        const double r = distanceFromCenter(d, x, y);
        double h = (r < t.craterRadius) ? craterFloor : coneHeight(t, r);
        h += amplitude * smoothNoise(seed, x, y);
        const int cells = static_cast<int>(std::ceil(h));     // skała to z < h
        hm.height[x + nx * y] = std::clamp(cells, 0, d.nz);
    }
}
```

(Fragment `TerrainGenerator::cone()` z uproszczonym rzutowaniem indeksu.)

Krok 5. Kratery (lava) albo zarodki (classic)

Co się dzieje: kratery to pierwsze komórki powietrza nad terenem: `placement = crater`: losowe kolumny w kraterze szczytowym, `random`: na zboczu, `positions`: w kolumnach z listy `vents.positions` (prawdziwe wulkany, bez losowania). Zarodki w trybie `classic` to losowe różne komórki całej przestrzeni. Losowanie robi `std::mt19937_64` z **własnym** losowaniem liczby z zakresu i własnym częściowym tasowaniem Fishera–Yatesa, bo `std::uniform_int_distribution` i `std::shuffle` dają różne wyniki w MSVC i GCC, a wyniki mają być identyczne na obu kompilatorach. Kratery dostają `flowId` 1… count i temperaturę erupcji.

Gdzie: `engine/src/terrain/VentPlacer.cpp`: `placeVents()` (11–58), `placeAt()` (60–81, lista kolumn), `placeSeeds()` (83–102); `Simulation::buildVents()` (`Simulation.cpp`, 148–156) wybiera wariant; `LavaCA::placeSeeds()` (`LavaCA.hpp`, 215–227); `uniformBelow()` w `core/Rng.hpp`.

```
std::mt19937_64 gen(splitmix64(seed ^ kSaltVents));
for (int i = 0; i < v.count; ++i) {   // częściowy Fisher-Yates: pierwsze count elementów =
wybór bez powtórzeń
    const std::size_t remaining = candidates.size() - static_cast<std::size_t>(i);
    const std::size_t j = i + uniformBelow(gen, remaining);
    std::swap(candidates[i], candidates[j]);
    vents.push_back(Vent{x, y, hm.at(x, y), i + 1});   // (x, y) = candidates[i]
}
```

(Fragment `placeVents()` uproszczony, bez rzutowań typów.)

Krok 6. Krok automatu komórkowego

Co się dzieje w jednym kroku (tryb lava): 1. **Warunki brzegowe**: ramka jest wypełniana przed krokiem: absorbujące = komórki `VOID` (nic nie wpływa, lawa przy brzegu „znika”), periodyczne = kopia przeciwległej warstwy. W trybie lava oś pionowa z jest zawsze absorbująca (grawitacja), w `classic` zawijane są wszystkie osie. 2. **D(n)**: dla każdej lawy leżącej na gruncie liczymy największy spadek dostępny wśród sąsiednich komórek powietrza („jak daleko w dół spadnie lawa, która tam wpłynie”). Osobny przebieg, bo D(n) jest potrzebne wielu sąsiadom. 3. **Reguły** dla każdej komórki: czytamy tylko `cur`, piszemy tylko `next` (**synchroniczny CA, podwójny bufor**): - komórka **powietrza** zbiera „kandydatów”, czyli sąsiadów-lawę, którzy mogą do niej wpłynąć: lawa **dokładnie nad nią** (swobodny spadek, zawsze), lawa **wyżej po skosie** (spływ po stoku, gdy obie mają podparcie i D = 1), lawa **na tym samym poziomie** (rozlewanie, tylko w kierunku największego spadku: spadek(c) = D(n)); lawa **niżej** nie wpływa (lawa się nie wspina), z wyjątkiem **wypełniania zagłębień** (komórka dokładnie nad lawą w niecce, poniżej poziomu przelewu, gdy lawa obok nie ma dokąd się rozlać); - zwycięża **najczęstszy** `flowId` wśród kandydatów (remis: `hashRand`), temperatura = max(temp kandydata − stygnięcie za ruch), `arrivalStep` = numer kroku; jeśli temperatura < `solidusTemp`, komórka zostaje powietrzem; - komórka **lawy**: krater w trakcie erupcji ma stałą `eruptTemp`, pozostała lawa stygnie o `coolingAmbient`; poniżej `solidusTemp` → **bazalt** (`solidStep` = krok); - skała i bazalt bez zmian. 4. **Koniec kroku**: zamiana buforów, historia (liczba komórek lawy, zmienione komórki), **warunek stopu**: brak płynnej lawy, brak zmian w kroku (tylko sąsiedztwa deterministyczne) albo `ca.maxSteps`.

Tryb **classic**: komórka powietrza z sąsiadem-ziarnem dostaje najczęstszy `flowId` sąsiadów (bez grawitacji i temperatury); koniec, gdy brak powietrza.

Sąsiedztwa to prekomputowane listy przesunięć (dx, dy, dz): von Neumann 4/6, Moore 8/26, losowe pentagonalne 5/17 (Moore bez jednej „strony", 4/6 wariantów), losowe heksagonalne 6 (Moore bez dwóch przeciwległych narożników, 2 warianty, tylko 2D). Wariant dla komórki w danym kroku wybiera `hashRand`.

Automat frontalny (domyślny, `ca.sweep = front`, z optymalizacji sekwencyjnej). Te same reguły, ale tylko dla komórek, które w danym kroku **mogą się zmienić**: komórek lawy i komórek powietrza, do których jakiś sąsiad-lawa (classic: nowe ziarno) może wpłynąć. Struktura: dla każdej warstwy osi pionowej (z; w przekroju 2D y) listy `src` (lawa; classic: ziarna z poprzedniego kroku), `cand` (kandydaci bez powtórzeń, znaczniki `mark_`), `nextSrc`, `changes`. Krok: `frontDescent()` (D(n) dla lawy) → `frontCandidatesRules()`: dla każdej warstwy najpierw kandydaci (`frontCandidatesLayer()`: komórki powietrza w zasięgu sumy przesunięć wszystkich wariantów sąsiedztwa), od razu reguły (te same funkcje reguł, czytają tylko `cur`, piszą listy warstwy; od wersji OpenMP jedna pętla po warstwach zamiast dwóch: kandydaci warstwy k zależą tylko od `src` warstw k − 1..k + 1, których reguły nie zmieniają) → `frontApply()` (zmiany do `cur`, kopie w ramce periodycznej `Boundary::syncCell()`). Wynik i historia liczników są **bitowo identyczne** z pełnym przebiegiem (`ca.sweep = full`, wersja referencyjna; testy `engine/tests/test_front.cpp`); szczegóły i pomiary: dokument „Optymalizacja wersji sekwencyjnej", punkt 3.4.

Gdzie: pełny opis reguł z uzasadnieniem każdej zmiany: `engine/src/ca/CaRules.hpp` (początek pliku); kod: `engine/src/ca/LavaCA.hpp`: `step()` (230–247), `stepLava()` (577–588), `stepClassic()` (591–601), `stepFront()` (685–692), `initFront()` (698–730), `frontStepSequential()` (761–781), `frontDescent()` (877–888), `frontCandidatesLayer()` (912–940; kandydaci jednej warstwy), `frontCandidatesRules()` (945–955; kandydaci i reguły zakresu warstw), `frontApply()` (1052–1066), `computeDescent()` (551–566), `updateLava()` (1109–1161), `gatherLavaCandidates()` (465–519), `tryPonding()` (443–461), `hasSupport()` (390–399), `dropDepth()` (404–414), `updateClassic()` (1164–1201), `finishStep()` (1206–1229); głosowanie `CandidateVote` (`CaRules.hpp`, 127–183); ramka `Boundary::apply()` (`engine/src/core/Boundary.hpp`, 66–73; wypełnianie: `applyWith()`, 75–131), `Boundary::syncCell()` (133–169); sąsiedztwa `engine/src/core/Neighborhood.cpp` (konstruktor, 55–119). Wersje wielowątkowe kroków: krok 6a.

```
// LavaCA::stepLava — jeden krok: ramka -> D(n) -> reguły (cur -> next) -> zamiana buforów
VOLCANO_NOINLINE void stepLava() {
    if (par_.parallel()) {                          // etap 9: wiele wątków — krok 6a
        finishStep(fullStepParallel(nullptr));
        return;
    }
    bc_.apply(cur_);
    computeDescent(0, layerCount());                // zakres warstw: tu wszystkie
    CandidateVote vote;
    StepCounts c;
    updateLava(stepContext(), 0, layerCount(), vote, c);
    finishStep(c);
}

// LavaCA::frontStepSequential<Strip> — automat frontalny (domyślny) w jednym wątku: tylko komórki, które mogą się
// zmienić (fragment; pełna wersja z pomiarem faz w LavaCA.hpp). Strip = false (engine): lo = 0, hi = nl — kod etapu 9;
// Strip = true (pas MPI, etap 10): warstwy własne [ownLo_, ownHi_), losowość z indeksem globalnym
if (p_.mode == CaMode::Lava) {
    frontDescent(lo, Strip ? descHi_ : nl);                  // D(n) tylko dla lawy (w pasie MPI także warstwa nad pasem)
}
frontCandidatesRules<Strip>(lo, hi, ctx, vote, sums);        // kandydaci warstwy + reguły (czytają cur_, piszą listy warstwy)
return frontCounts(sums, frontApply(lo, hi, periodic));      // zmiany do cur_ + kopie w ramce periodycznej

// LavaCA::updateLava — fragment dla komórki powietrza (pozycja p, sąsiedzi przez przesunięcia liniowe)
```

```
if (type == CellType::Air) {
    gatherLavaCandidates(x, y, z, p, prefix, idx, vote);
    if (!vote.empty()) {
        const int w = vote.winner(p_.seed, t, idx);          // najczęstszy flowId, remis ->
hashRand
        const float temp = vote.maxTemp(w);
        if (temp >= p_.solidusTemp) {
            next_.setCell(p, CellState{CellType::Lava, vote.flowId(w), temp, 0, stamp});
            ...
// LavaCA::gatherLavaCandidates — rozlewanie tylko w kierunku największego spadku
} else {
    if (cDrop != d || (d == 1 && diagonalFalls_)) {
        continue;
    }
    cooling = p_.coolingLateral;
}
```

Krok 6a. Wątki OpenMP

Co się dzieje: przy `parallel.threads > 1` (albo `--threads N`) krok automatu to **jeden region równoległy**, a fazy kroku są pętlami po **warstwach** (z; w przekroju 2D y) dzielonymi między wątki, z barierą między fazami: - automat frontalny: D(n) lawy | kandydaci + reguły | zmiany do `cur`; - pełny przebieg: ramka (ściany osi po kolei) | D(n) | reguły `cur` → `next`.

Każda faza pisze tylko elementy swojej warstwy (listy, znaczniki, komórki warstwy i ich kopie w ramce), a czyta stan z początku kroku albo wynik poprzedniej fazy (po barierze). Liczniki kroku są sumowane przez `reduction(+)`. Warstwa jest zawsze liczona w całości przez jeden wątek tą samą funkcją, więc wynik, historia liczników, a nawet kolejność elementów list są **bitowo takie same dla każdej liczby wątków i harmonogramu**. Przy 1 wątku kroki idą zwykłymi pętlami bez regionów OpenMP (ścieżka z optymalizacji sekwencyjnej).

Harmonogram (`parallel.schedule`, `parallel.chunk`) wybiera `ompFor()`: „osierocona" dyrektywa `omp for` wywoływana z wnętrza regionu. MSVC ma OpenMP 2.0 (bez `omp_set_schedule`), więc wariant dyrektywy wybiera `switch` w czasie działania. Przypinanie (`parallel.affinity = pcores`) ustawia przed obliczeniami jeden wątek na rdzeń fizyczny P (procesory logiczne 2, 4, …, 14, 0). Wyniki i pomiary: dokument „Zrównoleglenie OpenMP".

Gdzie: `engine/src/ca/LavaCA.hpp`: `fullStepParallel()` (606–673), `frontStepParallel()` (790–861), opis podziału na początku pliku (31–41; wersja MPI: 43–48); `engine/src/parallel/Omp.hpp`: `ParallelSettings` (80–87), `ompFor()` (93–143); `engine/src/parallel/Affinity.cpp`: `performanceCores()` (65–145), `pinThreads()` (147–181); `engine/src/parallel/ThreadStats.cpp`: `endStep()` (40–70, nierówność obciążenia), `toJson()` (72–121); `engine/src/core/Boundary.hpp`: `applyWith()` (65–121, ramka w pętli podanej przez wywołującego); testy `engine/tests/test_parallel.cpp`.

```
// LavaCA::frontStepParallel — jeden region na krok, warstwy dzielone między wątki (fragment)
#pragma omp parallel num_threads(par_.threads) reduction(+ : stay, changed, born, lava)
{
    if (lavaMode) {
        ompFor(nl, sched, chunk, [&](std::ptrdiff_t k) { frontDescent(k, k + 1); });
        ompBarrier();                          // reguły czytają D(n) źródeł z sąsiednich
warstw
    }
    ompFor(nl, sched, chunk, [&](std::ptrdiff_t k) { frontCandidatesRules(k, k + 1, ctx,
vote, sums); });
    ...                                        // sumy wątku -> reduction
    ompBarrier();                              // faza zmian modyfikuje cur_, który czytały
reguły
    ompFor(nl, sched, chunk, [&](std::ptrdiff_t k) { lava += frontApply(k, k + 1, periodic);
});
}
```

(Fragment uproszczony, bez rzutowań `k` na `int` i pomiaru czasu pracy wątków `--thread-stats`.)

Krok 6b. Procesy MPI: program `engine_mpi`

Co się dzieje: `mpiexec -n p engine_mpi --config …` startuje p kopii programu. Proces 0 wczytuje config (i teren) i rozsyła je pozostałym (`MPI_Bcast`), a każdy proces liczy **pas warstw** (te same warstwy z / y co wątki w kroku 6a) w swoim **oknie**: pas + warstwy brzegowe pod i nad nim (lawa 17 pod + 1 nad, classic i MC po 1). `LavaCA` i `PottsMC` dostają okno zamiast całej przestrzeni (`setStrip`) i liczą je tym samym kodem; różnice to trzy miejsca:

1. **przed każdym krokiem CA / po każdym kolorze MC**: `link_->exchange()`: warstwy brzegowe okna dostają aktualny stan warstw sąsiednich procesów (`MPI_Isend` / `MPI_Irecv` + `MPI_Waitall`, pakowanie pól warstwy do bufora; wariant `mpi.exchange = changed` wysyła tylko warstwy zmienione od ostatniej wysyłki);
2. **liczniki kroku** (lawa, powietrze, zmiany: reguła końca): `link_->sum()` = `MPI_Allreduce`, więc wszystkie procesy kończą w tym samym kroku z tą samą historią;
3. **losowość**: indeks globalny = indeks w oknie + `hashBase_` (pas jest ciągłym fragmentem tablic), więc `hashRand` daje te same liczby co w `engine` i wynik jest bitowo ten sam przy każdej liczbie procesów.

Na końcu proces 0 zbiera pasy wyniku (`MPI_Gatherv`) i zapisuje te same pliki co `engine` (+ sekcja `mpi` w `times.json`). Przy 1 procesie okno = cała przestrzeń, `link_ == nullptr`; kroki idą dokładnie ścieżką `engine` (szablon `frontStepSequential<bool Strip>`, krok 6). Opis i wyniki: dokument „Wersja rozproszona MPI".

Typy MPI i MPI-IO: trzy opcje (domyślnie jak dotąd, wynik bajtowo ten sam; dokument „Typy pochodne MPI i równoległy zapis"): `mpi.pack = datatype`: komunikat wymiany to typ pochodny (`MPI_Type_create_struct` z bloków „warstwa pola" pod adresami w siatce okna, wysyłka i odbiór od `MPI_BOTTOM`, bez bufora); `mpi.bcast = struct`: blok liczbowy configu i lista kraterów / zarodków jako struktury z procesu 0 (jeden `MPI_Bcast`, bez parsowania JSON); `mpi.io = mpiio`: każdy proces zapisuje swój pas do wspólnego pliku (widok `MPI_Type_create_subarray`, tekst: przesunięcia z `MPI_Exscan`) i czyta swój pas przy `--mc-from`, bez `MPI_Gatherv` / `MPI_Scatterv` na procesie 0.

Gdzie: `engine/src/mpi/main_mpi.cpp`: `main()` (20–75: `MPI_Init_thread` z `MPI_THREAD_FUNNELED`, kody wyjścia przez `MPI_Abort`); `engine/src/mpi/MpiSimulation.cpp`: `bcastText()` (53–68), `bcastConfig()` (76–124, JSON albo blok z typów MPI), `bcastVents()` (128–147), `bcastHeightmap()` (150–165), `gatherField()` / `scatterField()` (188–212), `ioJson()` (383–409, sekcja `mpi.io`), `setupProcess()` (446–475, priorytet i rdzenie P procesu), `prepareMcFrom()` (480–518), `MpiSimulation::checkConfig()` (530–547), `runCa()` (549–718), `runMc()` (720–1018), `runFromCli()` (1020–1393; odczyt pasów `--mc-from` przez MPI-IO 1126–1136, zapis CA przez MPI-IO 1163–1201); `engine/src/mpi/MpiHalo.cpp`: `sum()` (55–61), `exchange()` (89–102), `run()` (114–216: odbiory, pakowanie, wysyłka, `MPI_Waitall`, rozpakowanie), `messageType()` (218–265, typ komunikatu z adresów pól, typy MPI), `runTyped()` (281–410); `engine/src/mpi/MpiTypes.cpp`: `configBlockType()` (46–70), `ventType()` (72–80), `toBlock()` / `fromBlock()` (82–186); `engine/src/mpi/MpiIo.cpp`: `writeStrips()` (74–144, widok pliku i typ pamięci), `writeText()` (148–166, `MPI_Exscan`), `readCaStrip()` (208–237); `engine/src/parallel/Strip.cpp`: `splitLayers()` (8–20), `ghostsOf()` (34–67), `planStrip()` (71–120); `engine/src/ca/LavaCA.hpp`: `setStrip()` (153–162), `reachBelow()` / `reachAbove()` (167–168), `step()` (230–249, wymiana przed krokiem), `exchangeGhosts()` (347–355), `finishStep()` (1206–1229, suma liczników); `engine/src/mc/PottsMC.cpp`: `setStrip()` (48–59), wymiana po kolorze w `sweepColors()` (459, 494); testy `engine/tests/test_strip.cpp`, `engine/tests/mpi/test_mpi.cpp`.

```
// LavaCA::step — wersja rozproszona: warstwy brzegowe okna = stan sąsiednich procesów z
początku kroku (fragment)
if (link_ != nullptr) {
    exchangeGhosts();  // link_->exchange(cur_, …) + listy src warstw brzegowych dla automatu
frontalnego
}
...                    // krok jak w engine, tylko na warstwach własnych [ownLo_, ownHi_)
// finishStep: liczniki kroku sumowane po procesach — reguła końca jest globalna
```

```
if (link_ != nullptr) {
    link_->sum(v, 3);  // MPI_Allreduce(MPI_SUM)
}

// MpiHalo::run — wymiana warstw brzegowych (fragment)
MPI_Irecv(recvBuf_[i].data(), cap, MPI_BYTE, m.peer, tag_, comm_, &requests_[i]);       // od
każdego dostawcy warstw
...                                                                                      //
pakowanie: pola warstwy memcpy
MPI_Isend(buf.data(), size, MPI_BYTE, m.peer, tag_, comm_, &requests_[nr + j]);          //
do każdego odbiorcy
MPI_Waitall(nr + ns, requests_.data(), MPI_STATUSES_IGNORE);                            //
potem rozpakowanie do okna
```

Krok 7. Losowość z hasha: dlaczego wynik nie zależy od reprezentacji siatki

Co się dzieje: w kroku CA nie ma generatora ze stanem (`rand()`, `mt19937`). Każda losowa decyzja (remis w głosowaniu, wariant sąsiedztwa losowego) to `hashRand(seed, krok, globalnyIndeks, sól)`: kaskada funkcji mieszającej splitmix64. Wynik zależy **tylko** od ziarna, numeru kroku i położenia komórki (indeks `x + nx·(y + ny·z)` bez ramki), a nie od kolejności przetwarzania. Dlatego `VectorStorage` i `MatrixStorage` dają bitowo ten sam wynik (sprawdzają to testy i `--benchmark`), GCC i MSVC dają te same pliki, a wersja wielowątkowa (wersja OpenMP, krok 6a) daje ten sam wynik dla 1, 2, 3, 4 i 8 wątków, a w MPI dla każdej liczby procesów. „Sole" rozdzielają strumienie różnych decyzji.

Gdzie: `engine/src/core/Rng.hpp`: `splitmix64()`, `hashRand()` (46–52), sole `kSalt*`; użycie m.in. `CandidateVote::winner()` (`CaRules.hpp`, 152–173), `Neighborhood::variantFor()` i `variantFromPrefix()` (`Neighborhood.hpp`; stała część hasha liczona raz na krok: `hashPrefix()`/`hashIndex()` w `Rng.hpp`, 66–71, optymalizacja sekwencyjna).

```
inline std::uint64_t hashRand(std::uint64_t seed, std::uint64_t step, std::uint64_t
globalIndex,
                              std::uint64_t salt) {
    std::uint64_t h = splitmix64(seed ^ 0xA0761D6478BD642FULL);
    h = splitmix64(h ^ step);
    h = splitmix64(h ^ globalIndex);
    return splitmix64(h ^ salt);
}
```

Krok 8. Monte Carlo (model Pottsa)

Domena: tryb `basalt`: komórki bazaltu (i ewentualnie pozostałej płynnej lawy) z wyniku CA w pamięci albo z pliku (`--mc-from`); tryb `full`: cała przestrzeń, bez terenu i CA. MC ma własne płaskie tablice z ramką: orientacja `int32`, maska `uint8` (w domenie / przy ścianie periodycznej), mobilność `float`. Sąsiad to indeks + stałe przesunięcie liniowe, więc w pętli prób nie ma sprawdzania zakresu.

Inicjalizacja: każda komórka domeny dostaje orientację `1 + hashRand(seed, 0, idx, kSaltMcInit) mod Q`, powstaje lista komórek domeny, ramka jest wypełniana wg warunków brzegowych.

Mobilność ze stygnięcia: czas w stanie płynnym `solidStep - arrivalStep` (z CA) normalizowany min–max po domenie → `m = mobilityMin + (1 - mobilityMin)·norm`. W próbie komórka o mobilności m jest pomijana z prawdopodobieństwem 1 − m: długo płynna lawa przebudowuje się swobodnie (duże kryształy), szybko zastygła prawie wcale (drobne; „chilled margin" w skałach wulkanicznych).

Krok MCS = N prób, kroki 1–5 wersji naiwnej (oznaczone komentarzami w kodzie): 1. losowa komórka z listy domeny (ze zwracaniem), 2. energia przed zmianą `E = J · Σ(1 - δ(S_i, S_j))`: J razy liczba sąsiadów **w domenie** o innej orientacji, 3. nowa orientacja z całego zbioru 1…Q, różna od obecnej, 4. energia po zmianie, ΔE, 5. akceptacja Metropolisa: ΔE ≤ 0 zawsze, ΔE > 0 z p = exp(−ΔE/kT) (przy domyślnym kT = 0 nigdy, jak w wersji naiwnej). Po zaakceptowanej zmianie komórki przy ścianie periodycznej odświeżamy jej kopie w ramce (`syncFrame`).

Statystyki (co `statsEvery` MCS, poza czasem `mcSim`): liczba ziaren = spójne składowe komórek o tej samej orientacji (union-find; spójność von Neumanna dla MC z VN, Moore'a dla pozostałych), średnia

wielkość ziarna, energia całkowita, współczynnik akceptacji → `mc_stats.csv`; w `stats.json` także efekt mobilności (`byMobility`). Od optymalizacji sekwencyjnej: etykiety ziaren dla komórek wnętrza przez przesunięcia liniowe (bez zawijania współrzędnych), energia całkowita z par sąsiadów „do przodu", te same wartości, ok. 2× szybciej. Od wersji OpenMP energia i mapa kryształów liczą się w wątkach (sumy całkowite przez redukcję), a etykiety ziaren (union-find) zostały sekwencyjne.

Szachownica (`mc.update = checkerboard`, wersja OpenMP). Wariant naiwny nie daje się podzielić między wątki (każda próba widzi poprzednie). Szachownica dzieli komórki na **kolory**, z których żadne dwie komórki jednego koloru nie są sąsiadami: von Neumann: 2 (parzystość x + y + z), Moore: 4 w 2D i 8 w 3D, sąsiedztwa losowe jak Moore. Jeden MCS to kolory po kolei, w kolorze każda komórka domeny dokładnie raz (bez kroku 1, czyli losowania komórki), losowość `hashRand(seed, MCS, globalIndex, sól)`. Komórki koloru dzielą się między wątki po warstwach, z barierą po kolorze; wynik nie zależy od liczby wątków. To inny proces losowy niż wariant naiwny: walidacja statystyczna (ta sama kinetyka, różnice 1–2,4%): dokument „Zrównoleglenie OpenMP", punkt 3.

Gdzie: opis reguł: `engine/src/mc/McRules.hpp`; `engine/src/mc/PottsMC.cpp`: `initialize()` (104–143), `colorOf()` (153–158) i `buildColors()` (160–224; kolory i lista domeny kolor po kolorze, warstwa po warstwie), `fillFrame()` (226–277), `syncFrame()` (279–310), **`sweepTrials()` (313–391; pętla prób)**, `colorTrials()` (394–448) i `sweepColors()` (451–514; szachownica), `sweep()` (516–533), `measure()` (535–553), `totalEnergy()` (590–664); `unlikeNeighbors()` (174–186) i `unlikeBeforeAfter()` (191–204, energia przed i po zmianie w jednym przejściu, bez skoku warunkowego, z optymalizacji sekwencyjnej) w `PottsMC.hpp`; faza MC: `engine/src/app/McStage.cpp`, `McStage::run()` (93–301), `McStage::save()` (303–324); mobilność `engine/src/mc/CoolingMobility.cpp`, `fromCooling()` (76–83); ziarna `engine/src/mc/Grains.cpp`, `Grains::label()` (33–150); samo MC z pliku: `Simulation::runMcFrom()` (334–402).

```
const std::uint64_t prefix = hashPrefix(p_.seed, t);       // stała część hashRand(seed,
krok) — raz na MCS
for (std::uint64_t a = 0; a < n; ++a) {                    // 1 MCS = N_domena prób
    const std::uint64_t h = hashIndex(prefix, a);          // = wspólna część hashRand(seed,
t, a, ·)
    // Krok 1: losowa selekcja elementu (ze zwracaniem)
    const std::size_t i = domainFrame[hashSalt(h, kSaltMcPick) % n];
    if constexpr (kMobility) { if (toUniform(hashSalt(h, kSaltMcMobility)) >= mobility_[i])
continue; }
    const std::int32_t current = spin[i];
    // Krok 3: nowa orientacja z 1..Q, różna od obecnej (nie zależy od energii — liczona
przed krokiem 2)
    std::int32_t proposed = 1 + hashSalt(h, kSaltMcOrient) % otherStates;
    if (proposed >= current) ++proposed;
    // Kroki 2 i 4: energia przed i po zmianie w jednym przejściu po sąsiadach, bez skoku
warunkowego
    unlikeBeforeAfter(spin + i, current, proposed, offsets, count, unlikeBefore,
unlikeAfter);
    const double deltaE = p_.J * unlikeAfter - p_.J * unlikeBefore;
    // Krok 5: Metropolis
    bool accept = deltaE <= 0.0;
    if (!accept && p_.kT > 0.0) accept = toUniform(hashSalt(h, kSaltMcAccept)) < std::exp(-
deltaE / p_.kT);
```

(Fragment uproszczony; pełna wersja z komentarzami w `PottsMC.cpp`, `sweepTrials()`. Liczby losowe są te same co `hashRand(seed, t, a, sól)` przed optymalizacją, więc wynik bitowo identyczny; szablon `sweepTrials<kMobility, kRandomNb>` wybiera wariant pętli raz na MCS.)

Krok 9. Zapis wyników i pomiar czasu

Co się dzieje: wynik CA (wnętrze siatki bez ramki, `Microstructure`) zapisujemy jako: - `ca_result.txt`: linia `nx ny nz`, nagłówek `x y z type flowId temp`, potem komórka po komórce (x najszybciej), - `ca_result.bin` (wersja 2): `"VOLC"`, `uint32 2`, `int32 nx ny nz`, tablice `uint8 type[N]`, `int32 flowId[N]`, `float temp[N]`, `uint16 solidStep[N]`, `uint16 arrivalStep[N]` (13 B/komórkę, ~20× szybszy zapis niż txt), - wynik MC: `mc_result.txt` / `mc_result.bin` (`"VMC1"`, `int32 crystal[N]`),

`mc_stats.csv`, - `config_used.json`, `stats.json` (objętość lawy na krater, zasięg, historia kroków, sekcja `mc`), - `times.json`: czas każdej fazy [ms] (`readConfig, allocate, terrain, vents, caSim, caStats, save, mcInit, mcSim, mcStats, mcSave, total`; `caStats`: statystyki CA do `stats.json`, od wersji OpenMP), kroki, wymiary, procesor, kompilator, build, `memory` (szczyt pamięci procesu + model teoretyczny), `files` (rozmiar i czas zapisu każdego pliku), od wersji OpenMP także `threads`, `schedule`, `chunk`, `affinity`, `cpus` (procesory wątków przy `pcores`), `openmp` (wersja) i z `--thread-stats` `threadStats` (czas pracy każdego wątku w fazach kroku, nierówność obciążenia).

Każdą fazę mierzy `Timer` (`std::chrono::steady_clock`): zegar czytany na początku i końcu fazy, **nigdy w gorącej pętli** (stoper w pętli spowalniał krok lawy o ~10%, profil aplikacji).

Gdzie: `engine/src/io/ResultWriter.cpp`: `writeTxt()` (126–153; bufor dopisywany wskaźnikiem, temperatura przez `formatTemp()` (105–124) zamiast `printf`, optymalizacja sekwencyjna), `writeBin()` (155–167), `writeCrystal*()`, `binHeader()` / `crystalBinHeader()` (204–222, nagłówki wspólne z MPI-IO), `writeMcStatsCsv()`; `engine/src/io/ResultWriterSlabs.cpp`: `txtHeader()` (76–88), `appendTxtRows()` (90–116), `appendCrystalTxtRows()` (118–140): wiersze jednego pasa tym samym formatterem (zapis tekstu przez MPI-IO; osobny plik, żeby nie zmieniać kodu `writeTxt` generowanego przez MSVC); `engine/src/io/TimingReport.cpp`: `toJson()` (140–158), `recordMemory()` (104–138); `engine/src/core/Timer.hpp`; schemat pomiaru w `Simulation::runWith()`: każda faza to `t.reset(); …; timing.add("nazwa", t.elapsedMs());`.

```
// Simulation::runWith — wzór pomiaru każdego etapu
t.reset();
s.vents = buildVents(cfg_, hm);
ca.placeSeeds(s.vents);
timing.add("vents", t.elapsedMs());
...
t.reset();
ca.run();
timing.add("caSim", t.elapsedMs());

// ResultWriter::writeBin — nagłówek + surowe tablice (little-endian)
out.write(magic, 4);                          // "VOLC"
out.write(reinterpret_cast<const char*>(&version), sizeof(version));
out.write(reinterpret_cast<const char*>(dims), sizeof(dims));
writeArray(out, m.type);
writeArray(out, m.flowId);
writeArray(out, m.temp);
writeArray(out, m.solidStep);
writeArray(out, m.arrivalStep);
```

4. Jak działa GUI

```
przeglądarka (http://127.0.0.1:8000)          backend Python (gui/, FastAPI)
silnik C++ (osobny proces)
index.html + js/*.js  ──JSON / żądania──▶  app.py (endpointy /api)  ──podproces──▶  engine --
config … --out … --progress
viewer2d/3d, Chart.js ◀──bajty + X-Meta──  results.py, render.py    ◀──stdout PROGRESS──
                                           jobs.py (kolejka, workery) ──zapis──▶
configs/gui/…json, results/gui/…/, jobs.json
```

- **Backend nic nie liczy.** Formularz powstaje z `engine --print-defaults` (te same klucze co silnik), walidacja to `engine --validate`, symulacja: `engine` uruchomiony jako podproces (`gui/engine.py`). Serwer słucha tylko na 127.0.0.1, sprawdza nagłówek Host, POST tylko z JSON (`gui/app.py`, początek).
- **Po kliknięciu „Uruchom”** (`POST /api/jobs`, `gui/app.py` ~268 → `JobManager.create()`, `gui/jobs.py` 234–278): walidacja silnikiem (błąd → 422, nic nie powstaje) → zapis `configs/gui/<id>.json` → folder `results/gui/<id>/`
- `gui_job.json` → zadanie w kolejce. **Worker** (wątek, `_worker()` 535–553) bierze pierwsze oczekujące zadanie i w `_run()` (578–…) uruchamia `engine … --progress`: linie `PROGRESS ca 120/5000`

`active=1834` aktualizują postęp zadania, reszta wyjścia trafia do `engine.log`; na końcu status gotowe / błąd / anulowane. Stan zadań i kolejki jest w `results/gui/jobs.json` (zapis atomowy) i przeżywa restart.

- **Po kliknięciu „Uruchom serię”** (`POST /api/series`, `gui/app.py` ~343): `series.spec_from_dict()` + `series.expand()` (`gui/series.py` 339–405: iloczyn kartezjański osi, parowanie kluczy) → `JobManager.create_series()` (`gui/jobs.py` 292–…): walidacja **wszystkich** configów → `series.write_series()` (`gui/series.py` 434–470) zapisuje wszystkie `configs/gui/series/<id>/NN_<opis>.json` + `series.json` i tworzy wszystkie foldery `results/gui/series/<id>/NN_<opis>/` → dopiero potem wszystkie zadania trafiają naraz do kolejki. Kolejka ma regulowaną liczbę workerów, pauzę, ▲/▼, ponowienie, anulowanie serii i przypięcie procesu silnika do rdzenia P (`_pin_process()` 555–576: psutil / `SetProcessAffinityMask`, priorytet `HIGH_PRIORITY_CLASS`).
- **Przekrój 2D:** przeglądarka prosi `GET /api/result/slice?id&field&axis&index`; backend (`gui/results.py`, `ResultData.slice()` 465–487, `gui/render.py`, `slice_plane()` 49–55) wycina płaszczyznę z tablicy numpy (duże pliki `.bin` przez `np.memmap`: czytane są tylko potrzebne strony), próbkuje płaszczyzny > 4 mln komórek i wysyła **surowe bajty** (`application/octet-stream`) z opisem w nagłówku `X-Meta`. Przeglądarka tworzy z bajtów `TypedArray` i **sama koloruje** (`gui/static/js/palette.js`, `colorize()` 117–152; kolor kryształu z hasha ID, ten sam wzór co `tools/plot_slice.py`), rysuje na canvasie (`viewer2d.js`: `load()`, `draw()` 323).
- **Widok 3D** (three.js, `gui/static/js/viewer3d.js`): „ściany”: 6 ścian zewnętrznych bloku jednym żądaniem (`/api/result/faces`, `buildFaces()`), „powierzchnia”: najwyższa niepusta komórka każdej kolumny (`/api/result/surface`, `render.top_index()`, `buildSurface()`: siatka z wysokościami), „woksele”: lawa/bazalt jako `InstancedMesh` (`/api/result/voxels`, limit 150 tys.). Do przeglądarki nigdy nie idzie cały blok.
- **Porównanie** (`gui/compare.py`: `series_table()` 215–235, `result_row()` 90–143) czyta tylko `times.json`, `stats.json`, `config_used.json` każdego wyniku i składa tabelę; wykres rysuje Chart.js (`compare.js`, `renderCharts()` 221).
- **Wydajność** (dokument „Zakładka „Wydajność” w interfejsie”): jedyna zakładka bez wywołań silnika i bez endpointu danych: `tools/wydajnosc_gui.py` (`build()` 1910, sekcje `section_seq()` 645 … `section_tests()` 1539; zestawienie S / E z `bench/zestawienie.py`, pamięć z `bench/memory.py`) zapisuje plik statyczny `gui/static/data/wydajnosc.json`, a `gui/static/js/perf.js` (`PerfView`) pobiera go raz (`load()` 628) i rysuje: sekcja przy pierwszym wejściu (`showSection()` 685 → `buildSection()` 721 → `addCard()` 771: wykres, wniosek, „Jak czytać”, warunki, „Skąd dane”, link PDF), widok wykresu wg kompilatora i przełączników (`resolve()` 811: klucz „kompilator|opcja|…”, najbliższy widok, gdy brak pomiaru), konfiguracja Chart.js z widoku (`chartConfig()` 464, `datasets()` 349: kolor z roli serii przez `roleColor()` 78 i zmienne motywu), wtyczki wąsów min–max, etykiet, linii odniesienia i wykresu hantlowego (119–347), tabela (`tableHtml()` 545), eksport PNG (`exportPng()` 914). Dokument PDF do pobrania: `GET /api/doc/pdf?name=` (`gui/app.py`, `doc_pdf()` 624: tylko pliki z `docs/pdf/`).
- **Wygląd (dokument „Wygląd interfejsu”): skąd biorą się kolory**
- interfejs: zmienne CSS w `gui/static/css/style.css`: `:root` = motyw ciemny „bazalt”, `:root[data-theme="light"]` = jasny „popiół”; atrybut `data-theme` ustawia skrypt w `<head>` `index.html` (wybór z `localStorage` albo `prefers-color-scheme`), przełącznik w nagłówku;
- `gui/static/js/theme.js`: `themeColors()` (40) czyta zmienne CSS dla canvasu 2D, three.js i Chart.js, `setTheme()` (24) zmienia motyw i woła `onThemeChange()` (21); przekroje, scena 3D, wykresy, galeria i mapy rysują się od nowa; `applyChartTheme()` (75): domyślne kolory Chart.js; `algorithmColor()` (61): CA = lawa, MC = fiolet;
- kolory danych tylko w `gui/static/js/palette.js`: `TYPE_RGB` (powietrze i skała zależne od motywu: `setPaletteTheme()` 58), `FLOW_SLOTS` (35, kratery), `GLOW` (38, temperatura i „żar”), `rockShade()`

(105), `colorize()` (117), `legendFor()` (161), `showLegend()` (220; legenda podmieniana tylko przy zmianie, ważne dla płynności odtwarzania);
- 3D: przezroczysty canvas nad gradientem nieba z CSS, światło półkuli + „słońce" (`applyThemeLights()` 93 w `viewer3d.js`), emisja płynnej lawy w odtwarzaniu: `glowMaterial()` (700, atrybut `aGlow` ustawiany w `topColors()` 372 i `updateReplay()` 505).

5. Automatyzacja bez GUI

- **`engine --batch <folder> --out <folder>`** (`engine/src/app/Batch.cpp`: `BatchRunner::runFromCli()` 376–…, `collect()`, `run()` 186–…): wszystkie `*.json` z folderu w kolejności nazw (bez `series.json` i `_*.json`), **każdy w osobnym procesie silnika** (`core/ChildProcess.cpp`, `ChildProcess::run()`: `CreateProcessW` / `posix_spawn`), wynik do `<out>/<nazwa>/` + `engine.log`; błąd jednego nie przerywa serii; po każdym przebiegu `<out>/batch_summary.csv` (status, kroki, wszystkie czasy, pamięć, rozmiary plików); kod 5, gdy coś się nie udało. Osobny proces, bo szczyt pamięci procesu jest monotoniczny (pomiar jednego przebiegu) i awaria nie kończy serii.
- **`tools/make_series.py`**: ten sam generator co GUI (`gui/series.py`): `--template`, `--base`, `--vary`, `--range`, `--pow`, `--set`; zapisuje configi i foldery, a z `--run` woła `engine --batch` i wypisuje tabelę porównania (`gui/compare.py`). `--cpu 2`: przypięcie (dziedziczone przez `engine` i jego przebiegi).

5a. Prawdziwe wulkany i odtwarzanie erupcji

Szczegóły, wyniki i ograniczenia: **dokument „Prawdziwy teren i odtwarzanie erupcji"**.

Teren z prawdziwych danych (`tools/fetch_dem.py`): bbox (S, W, N, E) + zoom → kafelki Terrarium z AWS Terrain Tiles (`tile_range()` 91, `latlon_to_pixel()` 81, Web Mercator), dekodowanie `R·256 + G + B/256 - 32768` (`decode_terrarium()` 122), mozaika (`build_mosaic()` 189), środki komórek siatki → piksele → interpolacja dwuliniowa po wygładzeniu do rozmiaru komórki (`resample()` 236; **y = 0 to południe**, jak przy imporcie obrazów), metry → komórki: poziom morza / minimum → 0, szczyt → 0,85·nz (`to_cells()` 252), zapis CSV + PNG + JSON z metadanymi (`fetch()` 331). Silnik czyta CSV jak każdą mapę (`HeightmapLoader::loadCsv`), więc prawdziwy teren nie wymagał zmian w silniku. GUI czyta metadane (`gui/heightmap.py`, `load_geo()` 254) i pokazuje szer./dł. pod kursorem.

Kratery w zadanych kolumnach: `vents.placement = "positions"`, `vents.positions = [[x, y], …]`. Walidacja w `Config::fromJson()` (lista: `getColumns()` w `Config.cpp`, 111; liczba = `vents.count`), lista dla symulacji: `Config::ventColumns()` (435), rozmieszczenie: `VentPlacer::placeAt()` z `Simulation::buildVents()` (148). Lista celowo **nie jest polem struktury `Config`**: pole-wektor przesuwało w pliku wykonywalnym MSVC gorącą pętlę automatu (inne wyrównanie → CA wolniejsze o 6–9% przy identycznym kodzie automatu; analiza mapą linkera w `prawdziwe-wulkany.md`, punkt 7). GUI: panel „Kratery na mapie" (`gui/static/js/ventpicker.js`).

```
// Simulation::buildVents — kratery z listy bez losowania
if (c.vents.placement == VentPlacement::Positions) {
    return VentPlacer::placeAt(c.ventColumns(), c.space, hm);  // krater = (x, y, h(x, y)), flowId = numer na liście
}
```

Odtwarzanie erupcji krok po kroku (bez nowego wyjścia silnika). Krok t automatu czyta stan t i zapisuje stan t + 1, a nowa komórka lawy dostaje `arrivalStep = t`, zastygła `solidStep = t` (`LavaCA::updateLava`). Stan po k krokach:

typ końcowy	stan k
skała / powietrze	bez zmian (komórka nigdy nie była lawą)
lawa, bazalt	powietrze, gdy k < pojawienie (0 dla krateru, `arrivalStep + 1` dla reszty); bazalt, gdy k ≥ `solidStep + 1` (tylko typ końcowy bazalt); inaczej płynna lawa

Kratery mają `arrivalStep = 0` jak komórki zalane w kroku 0, więc rozpoznajemy je po położeniu (`stats.json` → `flows[].vent`). Temperaturę w kroku k liczymy dokładnie wstecz: płynna lawa traci `coolingAmbient` na krok, krater w trakcie erupcji ma `eruptTemp`, więc T_k = min(eruptTemp, T_końc + c·(koniec − k)).

Gdzie: `gui/replay.py`: `appear_steps()` (32), `solid_from()` (44), `state_at()` (50), `temp_at()` (60), `counts_per_step()` (67); `gui/results.py`: pola pochodne `appear` / `solidFrom` w `extract()` (294), `vents()` (325), `replay_info()` (355: kroki, parametry, liczby komórek płynnych/zastygłych w każdym kroku), `replay_cells()` (397: pakiet komórek lawy dla 3D, próbkowanie kolumn > 2 mln), migawki MC `mc_snapshot()` (441); endpointy `/api/result/replay` i `/api/result/replay/cells` (`gui/app.py`, 522–538). Przeglądarka (ta sama reguła co Python, test przez `node`): `gui/static/js/replay.js`: `stateAt()` (30), `tempAt()` (37), `colorizeReplay()` (58, paleta „żar" z `palette.js`), `ReplayBar` (119: suwak, Odtwórz/Pauza, spacja, prędkość, liczby komórek), `makeZip()` (292, klatki PNG), `recordWebM()` (344); 2D: `viewer2d.js`, `loadReplay()` (197) i `drawReplay()` (219); 3D: `viewer3d.js`, `replayData()` (311: dane raz, komórki posortowane wg pojawienia), `surfaceAt()` (345: powierzchnia w kroku k), `updateReplay()` (505: wysokości i kolory powierzchni / tekstury ścian / `count` wokseli, emisja płynnej lawy).

```
// gui/static/js/replay.js — stan komórki w kroku k (t = typ końcowy)
export function stateAt(k, t, appear, solidFrom) {
  if (t !== LAVA && t !== BASALT) return t;
  if (k < appear) return AIR;
  return k >= solidFrom ? BASALT : LAVA;
}
```

Migawki (narzędzie weryfikacji, domyślnie wyłączone): `output.snapshotsEvery` → `snapshots/step_<k>.bin` (stan 0, co N kroków, ostatni; `Simulation::runWith()`, 198–242; czas w sekcji `snapshots`, poza `caSim`) i `mc.snapshotsEvery` → `mc_snapshots/mcs_<k>.bin` (`McStage::run()`, 186–260; sekcja `mcSnapshots`, poza `mcSim`). Testy: `engine/tests/test_replay.cpp` (odtworzenie = stan automatu po każdym kroku, = migawki; migawki nie zmieniają wyniku), `gui/tests/test_replay.py`.

6. Benchmark vector vs macierz i profil

- **Co mierzy benchmark:** `engine --benchmark` liczy te same przypadki (rozrost classic 10^3…100^3, 100^2, 1000^2, lawa 2D i 3D) na `VectorStorage` i `MatrixStorage` naprzemiennie, `--repeats` razy, mediana czasu samego automatu, i porównuje wyniki komórka po komórce. Kod: `engine/src/app/Benchmark.cpp` (`runCase()` 145–…); tabela w `docs/profilowanie.md`, punkt 3.5.
- **Profil:** `bench/run_profile.py` (macierz pomiarów: rozmiary, zarodki, sąsiedztwa, kompilatory, przypięcie do rdzenia P, 3 powtórzenia) → `bench/results/profile_2026-10-09*.csv` (punkt odniesienia dla dalszej optymalizacji, nie nadpisywać), wykresy `bench/plot_profile.py` → `docs/img/profile_*.png`, profilery gprof/callgrind (`bench/profile_gcc.sh`) i Visual Studio (`bench/profile_msvc.py`) → `bench/profiles/`.

Główne wnioski: 1. **Macierz jest wolniejsza od płaskiej tablicy** w każdym przypadku (×1,05–1,67), najbardziej w dużym 2D; wyniki obu reprezentacji są bitowo zgodne (hash położenia). 2. **CA marnuje prawie całą pracę:** w lawie 68–83% czasu to przeszukiwanie sąsiadów komórek powietrza (`gatherLavaCandidates`), które w 99,99% nic nie znajduje; automat frontalny to główna planowana optymalizacja. 3. **Czas CA = N × kroki × 7–30 ns**, a kroki ∝ największa luka między zarodkami (∝ N^(1/d)) → czas rośnie jak N^1,2–1,6. 4. **MC: 60–67% czasu to liczenie sąsiadów o innej orientacji;** MSVC jest 2–3× wolniejszy od GCC przez jeden skok warunkowy w tej pętli (błędne przewidywania skoków); jedna linia kodu wyrównuje kompilatory. 5. **Model pamięci z kodu przewiduje szczyt procesu z dokładnością 0–4%:** ~40 B/komórkę (CA 3D), 91 B (CA 2D: ramka w nieużywanej osi), ~42 B (MC 3D); 1000^3 wymagałoby ~40 GB → argument za MPI (każdy proces trzyma tylko swój pas).

Optymalizacja sekwencyjna zrealizowała większość tej listy (automat frontalny, MC bez skoku warunkowego, ramka tylko w używanych osiach, przesunięcia liniowe, tańsze statystyki MC, szybszy

zapis txt): `etna_real` ×0,005, CA classic ×0,06–0,14, MC ×0,50–0,70, pamięć CA 2D 91 → 28–36 B/komórkę, wyniki bitowo te same. Narzędzia: `bench/ab.py` (test A/B), `bench/compare_results.py` (zgodność bitowa configów), `bench/plot_optymalizacja.py`; opis w dokument „Optymalizacja wersji sekwencyjnej".

Wersja OpenMP zrównolegliła automat i MC w OpenMP (krok 6a, szachownica w kroku 8): faza obliczeń przy 8 wątkach na rdzeniach P przyspiesza 4,8–6,7×, cały przebieg 2,1–6,2× (części sekwencyjne: teren, statystyki ziaren, zapis; prawo Amdahla), narzut przy 1 wątku ≤ 3%. Narzędzia: `bench/scaling.py` (S(p), E(p), Amdahl, harmonogramy, zgodność każdego powtórzenia), `bench/mc_validation.py`, `bench/plot_openmp.py`; opis w dokument „Zrównoleglenie OpenMP".

7. Kluczowe momenty kodu: ściąga

Co chcę pokazać / wyjaśnić	Plik i funkcja
wybór trybu programu, kody wyjścia	`engine/src/main.cpp`, `main()`
config: domyślne + plik + `--set`, walidacja	`engine/src/io/Config.cpp`, `Config::load()`, `fromJson()`
dwie reprezentacje siatki, ramka, SoA	`engine/src/core/Storage.hpp`, `VectorStorage::index()`, `MatrixStorage`
wypełnianie ramki (warunki brzegowe)	`engine/src/core/Boundary.hpp`, `Boundary::apply()`
sąsiedztwa (listy przesunięć, warianty losowe)	`engine/src/core/Neighborhood.cpp`, konstruktor `Neighborhood`
hashRand: losowość z położenia	`engine/src/core/Rng.hpp`, `hashRand()`
teren: stożek, szum, priority-flood	`engine/src/terrain/TerrainGenerator.cpp`, `cone()`, `fillDepressions()`
mapa wysokości z pliku	`engine/src/terrain/HeightmapLoader.cpp`, `loadCsv()`; GUI: `gui/heightmap.py`
rozmieszczenie kraterów (Fisher–Yates)	`engine/src/terrain/VentPlacer.cpp`, `placeVents()`
krok automatu, podwójny bufor, warunek stopu	`engine/src/ca/LavaCA.hpp`, `stepLava()`, `finishStep()`
automat frontalny (domyślny `ca.sweep = front`)	`LavaCA.hpp`, `stepFront()`, `frontStepSequential()`, `frontCandidatesRules()`, `frontApply()`; warunki zmiany komórki: `CaRules.hpp`
wątki OpenMP: krok w jednym regionie, fazy z barierami	`LavaCA.hpp`, `frontStepParallel()`, `fullStepParallel()`; `engine/src/parallel/Omp.hpp`, `ompFor()`
szachownica MC (kolory, MCS w wątkach)	`engine/src/mc/PottsMC.cpp`, `buildColors()`, `sweepColors()`, `colorTrials()`
przypinanie wątków do rdzeni P, czas pracy wątków	`engine/src/parallel/Affinity.cpp`, `performanceCores()`, `pinThreads()`; `ThreadStats.cpp`
skalowanie S(p), E(p), Amdahl, harmonogramy (OpenMP)	`bench/scaling.py`, `bench/plot_openmp.py`; walidacja szachownicy `bench/mc_validation.py`; wyniki `bench/results/omp/`
procesy MPI: pasy, warstwy brzegowe, wymiana, suma liczników	`engine/src/mpi/MpiSimulation.cpp`, `runCa()`, `runMc()`; `MpiHalo.cpp`, `run()`; `parallel/Strip.cpp`, `planStrip()`; `LavaCA::setStrip()`, `exchangeGhosts()`
skalowanie MPI, hybryda, MPI vs OpenMP	`bench/scaling_mpi.py`, `bench/plot_mpi.py`; zgodność `bench/compare_results.py --launch`; wyniki `bench/results/mpi/`
typy pochodne MPI: komunikat warstw z pól SoA, blok configu, tablica kraterów	`engine/src/mpi/MpiHalo.cpp`, `messageType()`, `runTyped()`; `engine/src/mpi/MpiTypes.cpp`, `configBlockType()`, `ventType()`; `MpiSimulation.cpp`, `bcastConfig()`, `bcastVents()`
MPI-IO: pas w pliku (subarray), tekst (`MPI_Exscan`), odczyt `--mc-from`	`engine/src/mpi/MpiIo.cpp`, `writeStrips()`, `writeText()`, `readCaStrip()`; pomiar `bench/mpi_io.py`, wyniki `bench/results/mpi_io/`
testy wydajności: zestawienie S / E, skalowanie słabe, pamięć, pamięć podręczna	`bench/zestawienie.py`, `scaling.py` / `scaling_mpi.py --weak` (`weak_cases()`), `memory.py`, `cachegrind.py`, `cell_time.py`, `micro/bandwidth.cpp`, `plot_testy.py`; wyniki `bench/results/testy/`
sąsiad przez przesunięcie liniowe	`engine/src/core/Storage.hpp`, `linear()`, `shifted()`, `nextX()`; `LavaCA::nbSteps_`
reguła rozlewania lawy (kierunek największego spadku)	`LavaCA.hpp`, `gatherLavaCandidates()`, `computeDescent()`, `descent()`
wypełnianie krateru (ponding)	`LavaCA.hpp`, `tryPonding()`; opis w `CaRules.hpp`
głosowanie o `flowId` (najczęstszy sąsiad)	`engine/src/ca/CaRules.hpp`, `CandidateVote::winner()`
kroki 1–5 MC (Metropolis)	`engine/src/mc/PottsMC.cpp`, `PottsMC::sweepTrials()` (wywołuje `sweep()`)
energia granicy ziaren	`engine/src/mc/PottsMC.hpp`, `unlikeNeighbors()`, `unlikeBeforeAfter()` (przed i po zmianie naraz)

Co chcę pokazać / wyjaśnić	Plik i funkcja
mobilność ze stygnięcia	`engine/src/mc/CoolingMobility.cpp`, `fromCooling()`
liczenie ziaren (union-find)	`engine/src/mc/Grains.cpp`, `Grains::label()`
zapis bin / txt	`engine/src/io/ResultWriter.cpp`, `writeBin()`, `writeTxt()`, `formatTemp()`
test A/B i zgodność bitowa programów (optymalizacja sekwencyjna)	`bench/ab.py`, `bench/compare_results.py`; wyniki `bench/results/ab/`
pomiar czasu i pamięci	`engine/src/app/Simulation.cpp`, `runWith()`; `engine/src/io/TimingReport.cpp`
porównanie vector / macierz	`engine/src/app/Benchmark.cpp`, `Benchmark::runCase()`
MC na wyniku CA z pliku	`engine/src/app/Simulation.cpp`, `runMcFrom()`; `engine/src/io/ResultReader.cpp`
seria w wersji serwerowej	`engine/src/app/Batch.cpp`, `BatchRunner::run()`; `engine/src/core/ChildProcess.cpp`
kolejka zadań, workery, przypięcie	`gui/jobs.py`, `JobManager._worker()`, `_run()`, `_pin_process()`; priorytet i przypięcie zadań wielowątkowych: `launch_mode()`
generator serii (iloczyn, parowanie, zapis plików)	`gui/series.py`, `expand()`, `write_series()`
tabela i wykres CA vs MC	`gui/compare.py`, `result_row()`; `gui/static/js/compare.js`, `renderCharts()`
przekrój 2D / widok 3D	`gui/results.py`, `ResultData.slice()`; `gui/static/js/viewer2d.js`, `viewer3d.js`
eksport VTK	`gui/export.py`, `write_vti()`; `tools/export_vtk.py`
prawdziwy teren (kafelki, przepróbkowanie, skala)	`tools/fetch_dem.py`, `fetch()`, `resample()`, `to_cells()`
kratery w zadanych kolumnach	`engine/src/io/Config.cpp`, `Config::ventColumns()`; `VentPlacer::placeAt()`; `Simulation::buildVents()`
odtwarzanie erupcji: stan w kroku k	`gui/replay.py`, `state_at()`; `gui/static/js/replay.js`, `stateAt()`, `tempAt()`
odtwarzanie w 3D (powierzchnia w kroku k)	`gui/static/js/viewer3d.js`, `surfaceAt()`, `updateReplay()`
migawki CA / MC	`Simulation::runWith()` (gałąź `snapshotsEvery`); `McStage::run()` (`writeSnapshot`)
wygląd GUI: motyw, kolory interfejsu i danych	`gui/static/css/style.css` (zmienne); `gui/static/js/theme.js`, `themeColors()`; `gui/static/js/palette.js`, `colorize()`
zakładka „Wydajność": dane i wykresy	`tools/wydajnosc_gui.py`, `build()`; `gui/static/data/wydajnosc.json`; `gui/static/js/perf.js`, `PerfView.render()`, `chartConfig()`

8. Słowniczek

Pojęcie	Znaczenie
komórka (woksel)	najmniejszy element przestrzeni, sześcian; ma typ (powietrze, skała, lawa, bazalt) i pola (`flowId`, temperatura…)
siatka 2D / 3D	2D: dokładnie jeden wymiar = 1 (lawa: przekrój x–z, `ny = 1`); 3D: blok `nx × ny × nz`, oś z pionowa
zarodek / ziarno	w CA: krater (lawa) albo zarodek (classic) z własnym `flowId`; ziarno = komórki o tym samym `flowId` / orientacji
sąsiedztwo	które komórki są „obok": von Neumann (przez ściany: 4 w 2D / 6 w 3D), Moore (także krawędzie i narożniki: 8 / 26), losowe pentagonalne (5 / 17), losowe heksagonalne (6, tylko 2D)
warunki brzegowe	co jest za brzegiem: absorbujące (nic, lawa znika), periodyczne (przeciwległy brzeg, „torus")
ramka / ghost cells	dodatkowa warstwa 1 komórki dookoła siatki; brzeg = wypełnienie ramki; w MPI: warstwy brzegowe okna procesu, kopie warstw sąsiednich procesów (lawa: 17 pod pasem, 1 nad)
pas, okno (MPI)	pas = ciągły zakres warstw z / y liczony przez jeden proces; okno = pas + warstwy brzegowe, z własną ramką
SoA	*structure of arrays*: osobna tablica na każde pole zamiast tablicy struktur; lepsze wykorzystanie pamięci podręcznej
synchroniczny CA, podwójny bufor	nowy stan wszystkich komórek liczony z kopii starego (`cur` → `next`), potem zamiana; kolejność komórek nie ma znaczenia
hashRand	„losowość bez stanu": liczba z funkcji mieszającej (ziarno, krok, indeks komórki, sól), ten sam wynik niezależnie od kolejności i podziału pracy
priority-flood	algorytm wypełniania zagłębień mapy wysokości kolejką priorytetową: poziom, do którego wypełni się niecka
D(n), spadek	największy spadek dostępny dla lawy w komórce n (ile poziomów w dół wyląduje lawa, która z niej wypłynie)

Pojęcie	Znaczenie
ponding	wypełnianie zagłębień: lawa podnosi się w kraterze warstwami do poziomu przelewu
solidus, solidStep, arrivalStep	temperatura zastygania; krok zastygnięcia i krok przybycia lawy do komórki (czas płynięcia = różnica)
model Pottsa	model ziaren: każda komórka ma orientację 1…Q, granica między różnymi orientacjami kosztuje energię J
MCS	*Monte Carlo step*: tyle prób, ile komórek w domenie MC
energia granicy	`E_i = J · Σ (1 - δ(S_i, S_j))`: J × liczba sąsiadów o innej orientacji
Metropolis	reguła akceptacji: $\Delta E \le 0$ zawsze, $\Delta E > 0$ z $p = \exp(-\Delta E/kT)$ ($kT = 0 \rightarrow$ nigdy)
mobilność	prawdopodobieństwo, że próba MC dla komórki w ogóle się odbędzie; z czasu płynięcia lawy
union-find	struktura do łączenia zbiorów, liczenie spójnych ziaren
times.json	plik z czasami faz [ms], pamięcią i rozmiarami plików jednego przebiegu
seria / oś / parowanie	zestaw zadań z iloczynu kartezjańskiego wartości parametrów zmiennych; oś = parametr(y) z listą wartości; parowanie = kilka kluczy w jednej osi zmienia się razem
worker	wątek GUI, który uruchamia zadania z kolejki (każde to osobny proces silnika)
rdzeń P / E	i9-13900HX ma rdzenie wydajne (P) i energooszczędne (E, 2,3–2,9× wolniejsze); pomiary przypinamy do P
region równoległy, bariera	fragment kodu wykonywany przez zespół wątków OpenMP; bariera: miejsce, w którym każdy wątek czeka na wszystkie pozostałe (następna faza czyta wynik poprzedniej)
harmonogram (`schedule`)	sposób podziału iteracji pętli między wątki: `static` (równe bloki z góry), `static,1` (cyklicznie po jednej), `dynamic` (pobierane w trakcie), `guided` (pobierane, porcje malejące)
szachownica (MC)	podział komórek na kolory bez dwóch sąsiadów tego samego koloru; komórki jednego koloru można aktualizować równolegle
przyspieszenie S(p), efektywność E(p)	$S(p) = T(1)/T(p)$, $E(p) = S(p)/p$: ile razy szybciej na p wątkach i jaka część idealnego przyspieszenia
prawo Amdahla, część szeregowa f	$S(p) = 1/(f + (1 - f)/p)$: część programu f, która się nie zrównolegla, ogranicza przyspieszenie do $1/f$
nierównomierność obciążenia	wątki dostają różną ilość pracy, a reszta czeka na najwolniejszy przy barierze (`--thread-stats`: Σ maks / Σ średnia czasu pracy wątku w kroku)
VTK `.vti`	format XML ImageData biblioteki VTK: regularna siatka z polami komórek, otwierany w ParaView
DEM, Terrarium	numeryczny model terenu (wysokości); Terrarium: kafelki PNG, w których wysokość jest zakodowana w kolorze R, G, B
bbox, zoom	prostokąt obszaru (szer./dł. S, W, N, E); poziom kafelków Web Mercator: piksel ~60 m (zoom 11) / ~29 m (zoom 12) na tych szerokościach
przewyższenie	ile razy skala pionowa komórek jest „gęstsza" od poziomej (Etna ×4,2: 234 m w poziomie, 55 m w pionie)
odtwarzanie erupcji	stan w kroku k liczony z `arrivalStep` / `solidStep` wyniku końcowego (bez ponownej symulacji)
migawka	zapis pełnego stanu CA po k krokach (`snapshots/step_<k>.bin`) albo kryształów po k MCS (`mc_snapshots/`)

9. Najczęstsze pytania (FAQ)

Dlaczego model wokselowy, a nie mapa wysokości (2.5D)? Wymagania (2D/3D, profil dla 10^3 i 100^3, wizualizacja 3D, sąsiedztwa i brzegi w 3D) pasują do pełnej przestrzeni 3D. Teren i tak jest mapą wysokości (kolumna skały), a lawa może tworzyć jeziora i spadać z urwisk, czego model 2.5D nie oddaje.

Jak lawa spełnia „rozrost ziaren"? Kratery = zarodki z ID, lawa rośnie do pustych komórek, nowa komórka dostaje najczęstsze ID wśród sąsiadów-kandydatów, wynik to mapa „która komórka należy do którego ziarna". Reguły fizyczne (grawitacja, kierunek spadku, stygnięcie) tylko ograniczają, gdzie ziarno może rosnąć.

Czemu macierz (`vector<vector<vector>>`) bywa szybsza dla lawy? W pierwszej wersji silnika bez przypięcia do rdzenia P lawa 64×64×48 wyszła na korzyść macierzy (×0,76–0,98): dane mieściły się w pamięci podręcznej, a płaska tablica liczy indeks z (x, y, z) dwoma mnożeniami przy każdym odczycie sąsiada. Po przypięciu i 5 powtórzeniach (profil aplikacji) macierz jest wolniejsza wszędzie (×1,05–1,67); tamten efekt był szumem pomiaru; koszt arytmetyki adresów (~29% instrukcji lawy) zostaje kandydatem do optymalizacji (przesunięcia liniowe).

Skąd różnica GCC / MSVC? W MC: MSVC kompiluje porównanie orientacji sąsiada do skoku warunkowego, który procesor często źle przewiduje (GCC używa `setne` bez skoku) i nie wyciąga stałej części hasha przed pętlę; 2–3× na małych siatkach. W CA lawy GCC 10–35% szybszy (arytmetyka adresów, niewstawione `offsetsFor`). Wyniki są bitowo identyczne; różni się tylko kod maszynowy (dokument „Profil aplikacji", punkt 2.4).

Jak MC zależy od sąsiedztwa? Więcej sąsiadów = droższa próba (Moore 3D: 26 sąsiadów × 2 przejścia), ale szybsze zgrubianie. W 2D (300×300, pomiar z dokumentu „Kolejka zadań, serie, dane wejściowe"): MC VN ~0,38 s, Moore ~0,52 s, hexagonal ~0,54 s na 100 MCS.

Czemu pentagonalne słabo rośnie? Wariant jest losowany co krok MCS i widzi 5 z 8 sąsiadów. Gdy pominięty „bok" zawiera komórki ziarna, zmiana orientacji ma $\Delta E = 0$ i jest akceptowana, więc małe ziarna ciągle się rozpadają (stan jak w wysokiej temperaturze; 40×40: 1 488 → 1 112 ziaren po 50 MCS, VN → 354). To cecha naiwnego MC z losowym wariantem, nie błąd.

Dlaczego MC jest wolniejsze od CA? Na tej samej siatce 4–56× (dokument „Kolejka zadań, serie, dane wejściowe", punkt 4.2: w 2D przewaga maleje z rozmiarem 25× → 4×, w 3D ~30×). Krok CA to jedno sekwencyjne przejście po komórkach (7–10 ns na komórkę), krok MCS to N losowych dostępów do pamięci, dwa przejścia po sąsiadach i kilka hashy na próbę (39–83 ns, rośnie, gdy dane wychodzą poza L2/L3). Do tego MC robi stałe 100 MCS, a CA kończy po 4–190 krokach. Do tego naiwny MC marnuje ~87% prób (akceptacja 13%). Losowy dostęp do pamięci sprawia też, że MC dużo mocniej zwalnia przy obciążonym komputerze (gra i inne programy w tle: MC 1000^2 2,8× dłużej, CA 1,5×).

Jak to jest zrównoleglone? CA jest synchroniczny (czyta `cur`, pisze `next` albo listy zmian) i ma losowość z hasha położenia: warstwy z dzielimy między wątki OpenMP bez zmiany wyniku (wersja OpenMP, krok 6a). MC w wariancie naiwnym jest sekwencyjne (każda próba widzi poprzednie), więc wątki liczą tryb szachownicy (kolory bez sąsiadów tego samego koloru), statystycznie równoważny wariantowi naiwnemu. W MPI (krok 6b) te same warstwy dostają procesy jako ciągłe pasy, a warstwy brzegowe okna są wymieniane przed każdym krokiem CA i po każdym kolorze MC; wynik jest bitowo ten sam przy każdej liczbie procesów (dokument „Wersja rozproszona MPI").

Co szybsze: MPI czy OpenMP? Na jednym komputerze z 8 rdzeniami P zwykle OpenMP: wątki dzielą warstwy dynamicznie i nie kopiują warstw brzegowych. MPI dorównuje mu tam, gdzie praca na warstwę jest równa (MC szachownicą, classic 2D: 7 procesów S = 6,7), a przegrywa w lawie (skupiona w kilku warstwach: stałe pasy mają nierówną pracę; Etna jest w MPI wręcz wolniejsza). Zaletą MPI jest pamięć: każdy proces trzyma tylko swój pas, więc siatka może być większa niż pamięć jednego komputera (dokument „Wersja rozproszona MPI", punkt 9).

Dlaczego 8 wątków przyspiesza mniej niż 8×? Faza obliczeń przyspiesza 4,8–6,7×: bariery między fazami kroku, nierówny podział (lawa skupia się w kilku warstwach), a ósmy wątek pracuje na procesorze 0 obsługującym przerwania systemu. Cały przebieg ograniczają części sekwencyjne (teren, etykiety ziaren MC, zapis): prawo Amdahla (dokument „Zrównoleglenie OpenMP", punkt 5).

Po co ramka? Żeby w gorącej pętli nie było `if`-ów na brzegach: sąsiad zawsze istnieje, a warunek brzegowy to tylko inne wypełnienie ramki. W MPI ramka w osi warstw = warstwy brzegowe okna (ghost cells).

Dlaczego `engine --batch` uruchamia osobne procesy? Szczyt pamięci procesu jest monotoniczny (w jednym procesie kolejny przebieg „widziałby" szczyt poprzedniego), awaria jednego przebiegu nie kończy serii, a czasy są takie same jak przy ręcznym uruchomieniu.

Skąd GUI zna stan erupcji w kroku k, skoro zapisujemy tylko wynik końcowy? Każda komórka lawy ma w wyniku krok przybycia i krok zastygnięcia; komórka przechodzi powietrze → lawa → bazalt tylko raz, a temperatura płynnej lawy maleje o stałe `coolingAmbient` na krok, więc typ i temperaturę w kroku k da się odtworzyć dokładnie. Sprawdzają to testy z prawdziwymi stanami automatu i migawkami silnika (`test_replay.cpp`, `test_replay.py`).

Czy symulacja Etny / Wezuwiusza jest realistyczna? Nie w sensie ilościowym: model nie jest skalibrowany (parametry dobrane pod czytelny obraz i czas ≤ 30 s). Kierunki spływu wynikają z rzeźby prawdziwego terenu (lawa schodzi stokami, Monte Somma zatrzymuje lawę Wezuwiusza), ale przy 234 m/komórkę lawa tworzy płaty, a nie wąskie jęzory: model wzrostu nie zachowuje masy, a doliny są płytsze niż jedna komórka pionowa (`prawdziwe-wulkany.md`, punkt 4).

Skąd wiadomo, że wyniki się nie zmieniły po zmianach w kodzie? Każda zmiana jest sprawdzana porównaniem `cmp` pliki `.bin` / `.txt` z programu sprzed zmian z nowymi (bitowo), testy sprawdzają zgodność vector/macierz i determinizm, a GCC i MSVC dają te same pliki.

Jak dobrane są parametry lawy? Model jest modelem wzrostu bez zachowania masy: zasięg wyznacza „budżet temperatury" (stygnięcie na krok i na ruch); parametry dobrałem do kształtu wyniku (jęzory, jezioro w kraterze), nie są fizyczne. Odniesienie: modele SCIARA (wypływ do najniższych sąsiadów).